

# Wiz Adoption Roadmap

---

**Status:** Draft

## Purpose

Bayview should adopt Wiz as a standard part of the software development lifecycle so developers can find and fix security issues earlier, security teams can maintain consistent visibility, and risky changes can be prevented before they reach production.

The goal is not only to enable another scanner. The goal is to make secure development easier by putting Wiz feedback where developers already work: in the IDE, in pull requests, and in CI/CD pipelines.

## Target State

- All active Bayview repositories are onboarded to Wiz Code or have a documented exception.
- All development teams use Wiz-supported IDE plugins where available so developers can scan and remediate before opening a pull request.
- All supported repositories run Wiz scans in CI/CD for applicable risk areas, including SAST, SCA, secrets, IaC, container images, and CI/CD posture where supported.
- Pull requests receive clear Wiz feedback, ideally as PR comments or annotations that summarize new findings, severity, policy impact, and remediation guidance.
- Required Wiz checks are included in branch protection for production-bound repositories.
- PR merge blocking is enabled in phases after policies are tuned, baselines are understood, and exception paths are available.
- Wiz findings are tracked against standard vulnerability management expectations, including ownership, severity, SLA, exception, and evidence requirements.

## Guiding Principles

- **Meet developers where they work.** IDE findings and PR comments should be the default feedback path before developers need to visit a separate dashboard.
- **Start with visibility, then enforce.** Teams need time to understand findings, remove false positives, tune policies, and fix existing debt before merge blocking is broadly enabled.
- **Block new risk before old risk.** Initial enforcement should focus on new critical issues, exposed secrets, high-confidence misconfigurations, and risky dependency changes introduced by a PR.
- **Use one standard path.** CI/CD templates, PR comment formats, policy thresholds, and exception handling should be consistent across Bayview.
- **Make exceptions visible.** Exceptions should be documented, time-bound, owned, and reviewed rather than handled as undocumented bypasses.
- **Prioritize by business risk.** Internet-facing, production-bound, customer-data, loan-data, privileged, and cloud-infrastructure repositories should move first.

## Adoption Phases

Phase 0: Foundation and Inventory

**Suggested timing:** Weeks 0-2**Objectives**

- Identify the repositories, teams, platforms, and pipelines that need Wiz coverage.
- Define the first version of Bayview's Wiz policies, thresholds, and ownership model.
- Prepare reusable onboarding materials so adoption can scale without one-off setup for every team.

**Actions**

- Build an inventory of active repositories, including business owner, technical owner, application name, data sensitivity, production exposure, and CI/CD platform.
- Group repositories into rollout tiers:
  - **Tier 1:** Production-bound, internet-facing, regulated, customer-data, loan-data, privileged, or cloud-infrastructure repositories.
  - **Tier 2:** Internal applications, APIs, scheduled jobs, and shared libraries.
  - **Tier 3:** Developer tools, scripts, prototypes, citizen-developed tools, and low-risk utilities.
- Confirm supported source control, CI/CD, and IDE environments used by Bayview developers.
- Define standard Wiz scan categories by repository type, such as SAST, SCA, secrets, IaC, containers, and CI/CD posture.
- Define initial policy thresholds for advisory reporting and future merge blocking.
- Create a standard exception process with owner, risk acceptance, expiration date, and compensating controls.
- Create developer quick-start guidance for IDE plugin setup, local scanning, PR review expectations, and finding remediation.

**Exit Criteria**

- Repository inventory exists and has rollout tiers.
- Standard CI/CD onboarding pattern is selected.
- Draft policies and future blocking thresholds are approved by AppSec and engineering leadership.
- Pilot teams are selected.

**Phase 1: Pilot and Developer Workflow****Suggested timing:** Weeks 3-6**Objectives**

- Prove that Wiz can provide useful developer feedback without creating excessive friction.
- Validate IDE plugin usage, PR comments, CI/CD templates, and policy tuning with a small set of teams.

**Actions**

- Onboard a representative set of Tier 1 and Tier 2 repositories.
- Install and authenticate Wiz IDE plugins for pilot developers using supported IDEs.
- Enable Wiz scans in CI/CD in non-blocking mode.
- Enable PR comments or annotations that summarize:
  - New findings introduced by the PR.
  - Severity and category.

- Whether the finding would violate future merge policy.
- Recommended remediation and links to additional detail.
- Track scan duration, false positives, developer feedback, and remediation outcomes.
- Tune policies and suppressions where findings are noisy, duplicated, or not actionable.
- Document the standard pipeline template and onboarding runbook.

### Exit Criteria

- Pilot repositories receive Wiz feedback in PRs.
- Pilot developers can run Wiz scans from supported IDEs.
- Scan duration and reliability are acceptable for normal PR workflows.
- AppSec and pilot teams agree on initial merge-blocking thresholds.

## Phase 2: Broad CI/CD Rollout

**Suggested timing:** Weeks 7-12

### Objectives

- Make Wiz scanning a normal part of Bayview CI/CD.
- Ensure every active development team sees Wiz findings before code merges or deploys.

### Actions

- Require Wiz CI/CD scanning for all Tier 1 repositories.
- Begin onboarding Tier 2 repositories using the standard template.
- Add Wiz PR comments or annotations to all onboarded repositories.
- Create dashboards for repository coverage, scan success rate, open findings, SLA status, exceptions, and trend reporting.
- Train engineering managers and tech leads on expected triage behavior.
- Require teams to identify owners for Wiz findings and remediation decisions.
- Create a recurring AppSec review of teams that are not yet onboarded, scans that are failing, and repositories without clear ownership.

### Exit Criteria

- 100 percent of Tier 1 repositories are scanning in CI/CD or have documented exceptions.
- At least 75 percent of Tier 2 repositories are scanning in CI/CD or have scheduled onboarding dates.
- PR comments or annotations are active for onboarded repositories.
- Findings are visible in a system of record and can be mapped to repository owners.

## Phase 3: Soft Enforcement

**Suggested timing:** Weeks 13-20

### Objectives

- Prepare teams for merge blocking without immediately stopping normal development.
- Make policy impact visible and predictable before enforcement begins.

### Actions

- Configure Wiz checks as required-to-run status checks for Tier 1 repositories.
- Keep checks advisory while showing whether the PR would pass or fail future policy.
- Require remediation or documented exception for newly introduced critical findings and exposed secrets.
- Start weekly reporting for policy violations that would have blocked a PR.
- Review recurring violations with engineering leadership and repository owners.
- Confirm break-glass and exception procedures for urgent production fixes.
- Publish the target date for required merge blocking.

### Exit Criteria

- Tier 1 repositories consistently run Wiz checks on PRs.
- Developers understand which findings will block future merges.
- Exception workflow has been tested and produces usable evidence.
- High-noise policies have been tuned or deferred.

## Phase 4: Merge Blocking for High-Risk Repositories

**Suggested timing:** Months 5-6

### Objectives

- Prevent high-confidence, high-impact issues from being merged into production-bound code.
- Establish merge blocking as a normal engineering control for high-risk repositories.

### Actions

- Enable PR merge blocking for Tier 1 repositories.
- Initial blocking thresholds should focus on:
  - New critical vulnerabilities.
  - Exposed secrets.
  - High-confidence critical or high IaC misconfigurations.
  - Critical vulnerable dependencies that are reachable or production-relevant.
  - Policy violations involving privileged cloud access, internet exposure, or sensitive data where Wiz context supports prioritization.
- Allow temporary exceptions only through the approved process.
- Monitor blocked PRs for development impact, false positives, repeated bypasses, and policy tuning needs.
- Publish guidance for common remediation patterns and frequently seen findings.

### Exit Criteria

- Tier 1 branch protection includes required Wiz checks.
- Blocked PRs have traceable remediation, exception, or risk acceptance outcomes.
- Engineering leadership receives recurring metrics on enforcement impact.

## Phase 5: Enterprise Enforcement and Optimization

**Suggested timing:** Months 7-12 and ongoing

### Objectives

- Extend consistent enforcement beyond the highest-risk repositories.
- Improve remediation speed, developer experience, and risk reduction over time.

## Actions

- Expand merge blocking to Tier 2 repositories after policies are tuned.
- Determine whether selected Tier 3 repositories require blocking based on data sensitivity, exposure, or operational importance.
- Increase enforcement thresholds over time as teams reduce existing vulnerability debt.
- Use Wiz reporting to identify systemic issues, such as recurring vulnerable libraries, insecure IaC patterns, hardcoded secrets, or weak CI/CD defaults.
- Add secure defaults to shared templates, starter repos, pipeline libraries, base images, and developer documentation.
- Review exceptions monthly and expire or renew them based on current risk.
- Use quarterly metrics to update policies, training, and engineering standards.

## Exit Criteria

- Wiz scanning and PR feedback are standard for all active Bayview repositories.
- Merge blocking is active for production-bound repositories and additional repositories based on risk.
- Exceptions are time-bound and reviewed.
- Wiz metrics are included in AppSec reporting and engineering governance.

# Minimum Adoption Requirements

## Developer IDE Usage

- Developers using supported IDEs must install and authenticate the Wiz plugin.
- Developers should run local or IDE-based scans before opening pull requests, especially for infrastructure, dependency, container, authentication, authorization, and secrets-related changes.
- IDE findings should be fixed before PR creation when the remediation is clear and low risk.
- Teams should track plugin adoption through available Wiz reporting, endpoint management, developer attestation, or team-level onboarding evidence.

## CI/CD Scanning

- Wiz scans must run automatically for supported repositories during CI/CD.
- Scans should run as early as practical, ideally on pull request creation or update.
- CI/CD scans should produce evidence that can be reviewed later, such as scan status, findings, timestamps, repository, commit, branch, and policy result.
- Pipeline failures caused by scanner availability should be monitored and handled through the standard break-glass process, not by permanently disabling scans.

## Pull Request Feedback

- Pull requests should include Wiz comments, annotations, or status checks that summarize security impact in developer-friendly language.
- Feedback should distinguish between pre-existing backlog and findings introduced or worsened by the current PR.

- Comments should link to detailed Wiz findings and remediation guidance.
- PR feedback should be concise enough that developers can act without sorting through a large vulnerability report.

## Merge Blocking

- Merge blocking should be phased in after teams have completed baseline onboarding and policy tuning.
- Blocking should begin with high-confidence risks that Bayview does not want to introduce into production.
- Required checks should be enforced through branch protection for production-bound repositories.
- Exceptions must be documented, time-bound, owned, and reviewed.

## Roles and Responsibilities

| Role                 | Responsibilities                                                                                          |
|----------------------|-----------------------------------------------------------------------------------------------------------|
| AppSec               | Define policies, thresholds, exceptions, reporting, developer guidance, and remediation expectations.     |
| Platform Engineering | Provide reusable CI/CD templates, branch protection patterns, and repository onboarding support.          |
| Engineering Managers | Ensure teams adopt IDE plugins, use PR feedback, remediate findings, and meet rollout milestones.         |
| Repository Owners    | Own findings, exceptions, remediation decisions, and evidence for their repositories.                     |
| Developers           | Run IDE scans, review PR feedback, fix introduced findings, and avoid bypassing Wiz controls.             |
| SOC / Cloud Security | Use Wiz context for investigation, detection, runtime correlation, and broader cloud risk prioritization. |
| Risk / Governance    | Review accepted risk, exception evidence, and adoption metrics where required.                            |

## Metrics

- Percentage of active repositories inventoried.
- Percentage of repositories onboarded to Wiz scanning by tier.
- Percentage of PRs with Wiz scan results.
- Percentage of PRs with Wiz comments or annotations.
- Percentage of supported developers with Wiz IDE plugin adoption evidence.
- Scan success rate and average scan duration.
- Number of new critical or high findings introduced by PRs.
- Number of blocked PRs by policy category.
- Number of exceptions opened, expired, renewed, or closed.
- Mean time to remediate by severity.
- Percentage of findings with clear repository and team ownership.
- Recurring findings by library, framework, IaC pattern, base image, or CI/CD configuration.

# Initial Policy Proposal

The first version of merge blocking should be conservative and focused on issues that are both high-impact and actionable.

| Stage               | Enforcement Mode                      | Suggested Policy                                                                                          |
|---------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Pilot               | Advisory                              | Show all findings and identify what would block later.                                                    |
| Broad rollout       | Advisory with required scan execution | Require scans to run, but do not block on findings except urgent cases approved by AppSec.                |
| Soft enforcement    | Advisory failure state                | Mark PRs as would-fail for new critical findings, exposed secrets, and selected high-confidence issues.   |
| Tier 1 blocking     | Blocking                              | Block new critical findings, exposed secrets, and selected high-confidence high or critical issues.       |
| Enterprise blocking | Blocking by risk tier                 | Expand blocking based on repository tier, data sensitivity, production exposure, and historical maturity. |

## Open Decisions

- Which source control and CI/CD platforms are in scope for the first rollout wave?
- Which IDEs are officially supported for mandatory plugin adoption?
- What is the exact threshold for blocking high findings versus critical findings?
- Where will exceptions and risk acceptances be recorded?
- Who owns central CI/CD templates and branch protection rollout?
- How will citizen-developed repositories be discovered and onboarded?
- What reporting cadence should be used for engineering leadership?

## References

- [Wiz Code](#)
- [Wiz CI/CD Security Scanning Guidance](#)
- [Wiz JetBrains IDE Plugin Announcement](#)